AI-Based Transformer Health Condition Monitoring

CAPSTONE PROJECT REPORT

Submitted by: [Student Name]

Register Number: [Register Number]

Department: Electrical and Electronics Engineering

College: [College Name]

Project Guide: [Guide Name]

Academic Year: 2026–2027

ABSTRACT

This project demonstrates an educational machine-learning workflow for classifying transformer condition as Normal, Warning, or High Risk using load, temperatures, voltage, oil level, moisture, vibration, and a hydrogen indicator. Since no real dataset was supplied, the package generates 1,800 clearly labelled simulated records. Results do not establish real-world predictive accuracy and must not be used for operational decisions.

1. INTRODUCTION

Transformers are essential power-system assets. Monitoring operating measurements can support investigation of abnormal conditions. This project demonstrates data preparation, exploratory analysis, model comparison, evaluation, and a dashboard in a reproducible prototype.

1.1 PROBLEM STATEMENT

Develop a prototype that uses transformer operating parameters to estimate a condition category, compares classification models, and displays the result while documenting assumptions and limitations.

1.2 OBJECTIVES

• Prepare and document data

• perform preprocessing and EDA

• train Random Forest and Gradient Boosting classifiers

• evaluate precision, recall, F1-score and confusion matrix

• show predictions in a Streamlit dashboard

• clearly state the non-clinical/non-certified educational status.

2. DATASET AND DATA DICTIONARY

The included dataset is simulated using a fixed random seed and transparent synthetic threshold rules. Features: load_pct (percent), ambient_temp_c (°C), oil_temp_c (°C), voltage_kv (kV), oil_level_pct (percent), moisture_ppm (ppm), vibration_mm_s (mm/s), dga_h2_ppm (ppm). Target: condition (Normal, Warning, High Risk). This is not measured field data or a public dataset.

3. METHODOLOGY

Generate CSV; load with Pandas; separate features and target; create stratified train/test split; impute missing numeric values using the median; scale numeric features; train Random Forest and Gradient Boosting; compare accuracy and weighted F1; save the best model and evaluation outputs; serve predictions through Streamlit.

4. SYSTEM DESIGN

Workflow: simulated CSV → preprocessing → train/test split → classifier comparison → evaluation metrics and plots → saved model → dashboard inputs → predicted condition category. Tools: Python, Pandas, NumPy, scikit-learn, Matplotlib, Seaborn, Joblib and Streamlit.

5. IMPLEMENTATION

Run `python generate_dataset.py`, then `python train_model.py`, then `python -m streamlit run app.py`. The training script saves a model in models/ and metrics, confusion matrix and feature importance in outputs/. The dashboard accepts operating parameters and displays a predicted category.

6. TESTING AND RESULTS

Verify dataset creation and columns; confirm training completes; check model_comparison.csv and classification_report.json; confirm confusion_matrix.png and feature_importance.png are created; launch the dashboard and test several input combinations. Insert the actual metrics generated by your run into the final submission; do not invent scores. Because labels are synthetic, metrics measure agreement with synthetic rules, not real transformer health.

7. SAFETY AND LIMITATIONS

The synthetic dataset cannot represent the full physics, sensor faults, ageing mechanisms, operational history, or maintenance records of real transformers. Class scores are not calibrated failure probabilities. Do not use the application to determine whether real equipment is safe or requires maintenance. Practical use requires validated field data, expert review and external evaluation.

8. FUTURE ENHANCEMENTS

• Use a documented public or permissioned dataset

• incorporate time-series sensor history

• perform external and time-based validation

• improve explainability and uncertainty reporting

• add secure monitoring integration only after engineering review.

9. CONCLUSION

The prototype demonstrates an end-to-end educational workflow for transformer condition classification, including data generation, preprocessing, model comparison, evaluation outputs, and a dashboard. Real-world usefulness remains unproven until tested on representative, validated data.

REFERENCES

1. Scikit-learn documentation: https://scikit-learn.org/stable/ 2. Pandas documentation: https://pandas.pydata.org/docs/ 3. Streamlit documentation: https://docs.streamlit.io/ 4. Matplotlib documentation: https://matplotlib.org/stable/ 5. Seaborn documentation: https://seaborn.pydata.org/ 6. Supplied capstone brief: “AI-Based Transformer Health Condition Monitoring.”

SUBMISSION CHECKLIST

Fill student name, register number, college, guide and date. Run the code and insert actual screenshots. Include actual metrics from outputs/. Include dataset source and data dictionary; clearly identify the dataset as simulated. Prepare a final PPT and viva explanation.